CareerHub — Website Flow & Data Sync

Screen-to-screen flow, real-time sync architecture, and the encrypted data lifecycle

This document ties the previous five together into one operational picture: what screen leads to what next, how data stays in sync in real time between the website, the extension, and the backend, and exactly where encryption applies at each step of a piece of data's life — from the moment it's typed in to the moment it's used to fill a form on another website.

1. End-to-End Website Flow (Screen Sequence)

The loop closes deliberately: every tool feeds back into the Dashboard and the Application Tracker, so the product feels like one connected workflow rather than a set of disconnected utilities — reinforcing the 'enter once, use everywhere' promise visually, not just technically.

2. Screen-by-Screen: What Comes Next & Why

From screen	Leads to	Why / rule
Landing Page	Sign up / Log in	First-time vs. returning users forks here; returning users skip straight to Dashboard.
Auth	Profile Setup (first-time) or Dashboard (returning)	Check profile-completeness flag server-side to decide the fork, not just 'is this a new account.'
Profile Setup	Dashboard	Setup can be exited early (partial profile) -- Dashboard then shows a completion nudge rather than blocking access.
Dashboard	Any tool (Resume Builder, ATS Checker, Cover Letter, Extension Setup)	Dashboard is the hub every tool returns to -- no tool is a dead end.
Resume Builder	Saved Resumes -> optionally 'Use with Extension'	Export always saves a variant record (see Profile Schema doc) before offering download.
ATS Checker	Suggested Fixes -> back into Resume Builder to apply them	Fixes are suggestions the user applies manually inside the builder -- checker never edits Profile directly.
Extension Setup	Extension Popup (on any supported portal tab)	Setup on the website just authenticates the extension; actual use happens contextually on portal pages.
Extension Popup -> Use	Application Tracker entry auto-created	Tracker entry created the moment fields are filled, not only after manual confirmation -- status field starts as 'Draft' until user confirms submission.
Application Tracker	Dashboard stats update	Real-time listener updates Dashboard counters without a page refresh (see Section 3).

3. Real-Time Sync Architecture

Three surfaces need to stay in sync: the website (any tab/device), the browser extension, and the backend. The design goal is that editing your phone number on the website is instantly reflected in the extension's next fill, with no manual refresh anywhere.

Sync mechanism:

- **Live listeners, not polling:** the website and extension both subscribe to real-time change streams on the user's Profile document (Firestore-style snapshot listeners, or WebSocket push) rather than periodically re-fetching.
- **Optimistic UI updates:** when a user edits a field, the UI updates immediately (assume success), while the write confirms in the background; on failure, the UI rolls back and shows an inline error -- never silently loses the edit.
- **Extension refresh trigger:** the extension keeps a lightweight local encrypted cache of the Profile for fast fills; a change event from the live listener invalidates and refreshes that cache in the background, so the next portal visit always uses current data.
- **Multi-tab consistency:** if the Profile is open in two browser tabs, both receive the same live update stream so neither tab shows stale data or silently overwrites the other's edit.
- **Conflict resolution:** last-write-wins at the field level (not whole-document level) -- editing 'phone' in one tab and 'skills' in another simultaneously does not cause either edit to be lost.

Sync flow diagram:

Real-time Dashboard stats:

Application Tracker writes (from an extension fill event) push to the same live-listener pipeline, so Dashboard counters ("applications this week", "profile completeness") update the instant an event happens elsewhere, without the user needing to be on the Dashboard tab at that moment for it to register server-side.

4. Data Lifecycle: Storing, Extracting & Using Data Securely

Every stage a piece of data passes through has a specific, deliberate security control. This traces one field (phone number) through its entire life as a concrete example:

Stage	What happens	Security control
1. Input	User types phone number into Profile Setup form	TLS in transit immediately; client-side format validation before send (not a security control, but prevents garbage data).
2. Transit to backend	Form submits the value to the API	TLS 1.2+ required; short-lived auth token (not permanent API key) authenticates the request.
3. At rest	Backend stores the value in the Profile document	AES-256 encryption at rest; field-level encryption for higher-sensitivity fields (IDs) even within the same document.
4. Read for display	Website re-renders the Profile Setup or Dashboard	Value decrypted server-side just-in-time for the authenticated request only; never decrypted in bulk/at-rest.
5. Sync to extension	Live listener pushes the change	Encrypted payload in transit; extension stores it in an encrypted local cache, not plaintext localStorage.
6. Extraction for autofill	Extension matches this field to a detected form input on a job portal	Decrypted only in memory at the moment of filling; never written to disk unencrypted, never sent to any third party.
7. Post-use	Fill completes, field logged in Application Tracker	Tracker stores a reference/summary ("applied via Naukri, 11 Aug"), not a duplicate copy of the raw field value.
8. Deletion	User deletes Profile or requests data export	Full export returns decrypted JSON to the user only (authenticated download); deletion purges the field from primary storage and scheduled backups within a stated window.

Principle: data is decrypted for the shortest possible time, in the smallest possible scope, and only in memory during active use -- never sitting decrypted in a cache, log file, or browser storage at rest.

5. Extension-to-Website Real-Time Link (Detail)

Since the extension and website are separate execution contexts, they need an explicit, secure channel rather than assuming shared state:

- **Auth handshake:** extension install links to the account via a one-time secure code shown on the website (similar to how desktop apps pair with mobile apps) -- avoids ever typing a password into the extension directly.
- **Token refresh:** the extension holds a short-lived token, silently refreshed via the backend while the user is active, and requires re-auth if unused for an extended period -- limits the blast radius if a device is ever compromised.
- **Push, not pull, for updates:** Profile edits push to the extension via the live-listener channel described in Section 3, so field-map updates and Profile edits both arrive the same way, keeping the sync model consistent across data types.
- **Fill events flow back:** when the extension fills a form, it pushes an Application Tracker event back to the same pipeline, which is what makes Dashboard stats update in real time without the user navigating back to the website at all.

6. Offline Handling & Failure Recovery

Scenario	Behavior
Extension has no network at fill-time	Use last-synced encrypted local cache to fill (slightly stale data is still useful); flag the fill as 'used cached data' in the confirmation chip so the user knows to double check.
Backend write fails after optimistic UI update	Roll back the optimistic change in the UI, show a clear inline retry prompt -- never leave the UI showing a value that wasn't actually saved.
Live listener disconnects (network drop)	Fall back to a manual refresh-on-reconnect; show a small 'reconnecting...' indicator rather than silently showing stale data as if it were live.
Two tabs edit the same field simultaneously	Field-level last-write-wins, with the losing tab's UI reconciled to the winning value once the listener delivers it -- no silent data loss, just a visible correction.
Extension cache is stale beyond a safety threshold (e.g. 7+ days unsynced)	Force a fresh fetch before allowing a fill, rather than risking a fill with badly outdated information.

Together, Sections 3-6 describe a system where the website, extension, and backend behave as one consistent product rather than three separately-synced pieces -- which is what makes the 'enter once, use everywhere, always current' promise actually true in practice, not just in marketing copy.